

Post-mortem analysis of an AI code-generation pipeline

Comparing `tradinGroups` (handwritten implementation) with the project generated by GitHub Copilot agents (`AppFromPrompts`)

Technical analysis report

July 12, 2026

Abstract

This document analyses the outcome of a software development experiment driven entirely by AI agents: rebuilding a financial web application (a stock-trading simulator with social groups) starting from a single specification document, through a pipeline of sequential prompts fed to GitHub Copilot. The generated result (repository `AppFromPrompts`) is compared against the handwritten reference implementation (repository `tradinGroups`). The analysis quantifies what worked (perfect type-safety, broad functional coverage, process discipline), what failed (81 out of 265 test suites do not pass, three parallel implementations of the same business logic, a queue architecture built but never wired, a last-minute improvised integration phase) and reconstructs the root causes, with particular attention to the mechanisms of *context rot* — the progressive degradation of coherence when multiple agents work in isolation on partial contexts. The document closes with operational recommendations for the next iteration of the pipeline.

Contents

1	Introduction and goal	2
2	The pipeline that was used	2
2.1	Phase 1 — Expanding the specification into four technical documents	2
2.2	Phase 2 — Decomposing each document into atomic features	2
2.3	Phase 3 — Consistency check of the feature documents	3
2.4	Phase 4 — Creating the MASTER_SYSTEM_CONTEXT	3
2.5	Phase 5 — Implementing each feature (one agent per feature)	3
2.6	Phase 6 — Per-feature QA (Gap Analysis)	3
2.7	Phase 7 — Integration tests per document	3
3	Analysis methodology	3
4	State of the generated project: the numbers	4
4.1	Overview	4
4.2	Compilation: green	4
4.3	Tests: red, even though the reports claim otherwise	4
4.4	Commit timeline	4
5	What worked	5
6	What went wrong	5

6.1	The triple implementation of the business logic	5
6.2	The queue architecture: built, tested, never wired	6
6.3	The API monolith and the app that ran on mocks	6
6.4	The breakdown of the failing tests	6
6.5	RBAC: the middleware exists, the routes do not use it	7
6.6	Documentation inflation and invented features	7
6.7	Anomalous project structure	8
6.8	Uneven QA coverage	8
7	Comparison with tradinGroups	8
7.1	Opposite philosophies	8
7.2	Key observations	8
8	Root-cause analysis: where context rot struck	9
8.1	Cause 1 — Agent isolation without executable shared contracts	9
8.2	Cause 2 — Integration as a final phase instead of a continuous constraint	9
8.3	Cause 3 — Local “green” mistaken for global “green”	9
8.4	Cause 4 — Documentation expansion as a hallucination multiplier	9
8.5	Cause 5 — Global rules losing against local instructions	10
9	Recommendations for the next iteration of the pipeline	10
10	Conclusions	11
A	Appendix A — Full test-suite outcome	12
B	Appendix B — Duplication inventory	12
C	Appendix C — Essential timeline of the generated project	12

1 Introduction and goal

The experiment was born from a precise question: *is it possible to recreate a real web application, already built by hand, using exclusively a pipeline of structured prompts and coding agents, without running into the classic context-rot failure modes?*

The two artifacts being compared are:

- **tradinGroups** — the handwritten reference implementation: Express + Prisma + PostgreSQL backend, Vite + React frontend (SPA with React Router), images on Cloudinary, trading engine driven by `setInterval`. About 16,800 lines of TypeScript, 30 commits between March 19 and April 4, 2026.
- **AppFromPrompts** — the project generated by the pipeline: a monorepo with an Express + Prisma backend and a Next.js frontend (App Router) nested inside `src/frontend`, about 23,500 lines of source code plus 26,400 lines of tests (265 test files), 384 commits between February 22 and March 19, 2026.

The synthetic verdict, up front: **the pipeline produced an enormous amount of code of good local quality, but not an integrated, end-to-end working application.** Each feature, taken in isolation, is well written, typed and tested; the system as a whole, however, contains three parallel implementations of the same business logic, an entirely dead queue infrastructure, and a hasty final integration phase that broke previously green tests without any agent noticing. Context rot did not manifest as “the agent forgets its instructions”, but as **systemic divergence between parallel worlds built by agents that never talked to each other.**

2 The pipeline that was used

The pipeline consisted of seven types of prompts, applied in sequence. A detailed description is given here, because the structure of the pipeline itself is part of the causes analysed later.

2.1 Phase 1 — Expanding the specification into four technical documents

A prompt with the role of “Senior Software Architect and Tech Lead” receives `specifich.md` (327 lines) and generates four “progressive” technical documents: (1) database architecture and construction, (2) backend engine and core algorithms, (3) API infrastructure and security, (4) frontend and UI/UX. The fundamental rule imposed was: *“DO NOT SUMMARIZE. Each document must be long, extremely detailed and include every single business rule”*.

Result: 4 documents totalling **11,154 lines** (respectively 1,337, 3,585, 2,329 and 3,903 lines) — an expansion factor of roughly 34× over the original specification.

2.2 Phase 2 — Decomposing each document into atomic features

Each document is broken down into “independent atomic features”, each with its own specification document intended as the system prompt for a TDD agent. Key requirements: a strict Red-Green-Refactor cycle, Conventional Commits at every green cycle, and above all *“each document must allow an agent to work without knowing the other features”*.

Result: **42 features in total** — 12 for the database document, 12 for the backend, 10 for the APIs, 8 for the frontend.

2.3 Phase 3 — Consistency check of the feature documents

A verification prompt asked whether, executing all features with one agent each, the complete app would emerge in an orderly and coherent way. For document 4 this phase actually spotted real problems (the `GroupCard` component triplicated across three features, undeclared dependencies between features 3, 5 and 8) and produced two corrective artifacts: `FIX_REPORT.md` and `SHARED_CONTRACTS.md`, the latter containing the shared TypeScript contracts. It is telling that these artifacts exist *only* for document 4: for documents 1–3 the verification left no equivalent traces.

2.4 Phase 4 — Creating the MASTER_SYSTEM_CONTEXT

A synthesis prompt condensed the specification + the 4 documents into a context file for the agents (`MASTER_SYSTEM_CONTEXT.md`, 329 lines): non-negotiable business rules, roles/permissions matrix, essential DB schema, endpoint reference, trading-engine flow, tech stack (with an explicit ban on Next.js’ Pages Router), error codes with the exact message strings, DO/NEVER guidelines — including the upper-case rule on **mandatory mocking** of queues, Redis and external APIs in tests.

2.5 Phase 5 — Implementing each feature (one agent per feature)

Each agent received as its “entire universe of knowledge” exclusively its own feature file plus the master context, with the explicit instruction to ignore any other convention. It had to commit at the start (to separate features), then at every green, self-verify at the end of the work and produce a report file (`RESOCONTO_FEATURE_*.md`).

2.6 Phase 6 — Per-feature QA (Gap Analysis)

A prompt with the role of “Senior QA Engineer” compared the implementation against the PRD document, producing a diagnostic report ordered by severity (BLOCKING / MAJOR / MINOR) on gaps, domain constraints, RBAC and edge cases.

2.7 Phase 7 — Integration tests per document

Once all the features of a document were complete, an “SDET” prompt generated an integration suite (Jest + Supertest + Prisma) covering flow isolation, domain errors, RBAC and mocking of external services. Each phase was followed by possible correction cycles if the checks failed.

3 Analysis methodology

The analysis was carried out directly on the two repositories, with reproducible measurements:

- **Compilation:** `npx tsc -noEmit` on the entire generated project.
- **Tests:** full run of `npx jest` (265 suites), classifying failures by cause (environment error vs real regression) through log inspection.
- **Git history:** temporal distribution of the 384 commits, adherence to Conventional Commits, “commit at every green” pattern.
- **Structural analysis:** mapping of services, repositories, workers, middleware and their import dependencies, to identify dead code and duplications.
- **Document comparison:** original specification vs generated documents vs code, plus the process artifacts (`TODO.md`, `VERIFICA_SPECIFICHE.md`, diagnostic reports, feature reports).

- **Comparison with tradinGroups:** Prisma schema, runtime architecture, frontend structure, cron/image handling.

4 State of the generated project: the numbers

4.1 Overview

Metric	AppFromPrompts (generated)	tradinGroups (manual)
TypeScript source lines	~23,556	~16,817
Test lines	~26,391 (265 files)	0
Commits	384 (Feb 22 – Mar 19)	30 (Mar 19 – Apr 4)
REST endpoints	68 (in a single file)	~60 (per domain, 5 routers)
Prisma migrations	12	13
Prisma models	11 + 5 enums	12 + 5 enums
DB triggers	subset (across 6 migrations)	dedicated migration
Frontend	Next.js App Router in <code>src/frontend</code>	Vite + React Router, separate package
Order queue	BullMQ/Redis (never wired)	<code>setInterval</code> 60s

Table 1: Quantitative comparison between the two projects.

4.2 Compilation: green

`tsc -noEmit` finishes with **0 errors** over roughly 50,000 total lines. This is a non-trivial result and deserves recognition: the strict TypeScript constraint imposed by the master context held across 42 different agents.

4.3 Tests: red, even though the reports claim otherwise

The full suite run yields:

81 suites failed / 183 passed / 1 skipped — **370 tests failed** / 589 passed out of 963

Every single `RESOCONTO_FEATURE_*.md` instead declares its own suite green, and the diagnostic report for document 1 (dated March 17) certifies “75 passed, 1 skipped”. This gap between *declared state* and *actual state* is the single most important finding of the entire analysis, and it is broken down in Section 6.4.

4.4 Commit timeline

The temporal distribution tells how the work actually went:

Day	Commits	Main activity
Feb 22–23, 2026	14	Frontend features (doc 4): profile, images, first Next.js skeleton
Mar 13, 2026	18	Resumption: friendship DB guards (doc 1)
Mar 16, 2026	23	Doc 1 features (stock, portfolio, transactions)
Mar 17, 2026	200	Bulk of doc 1–3 features + gap closure
Mar 18, 2026	128	Remaining features, wiring, TODO remediation
Mar 19, 2026	1	Final touch

Table 2: Commits per day in the generated repository.

Half of the project (328 commits out of 384) was generated in 48 hours. All integration decisions were taken in those same 48 hours — and that is where the damage is concentrated.

5 What worked

It is important to be honest about the successes, because they define what to *keep* of the pipeline.

1. **Total type-safety.** Zero compilation errors over 50k lines written by 42 different agents. The “TypeScript + Prisma for everything” constraint in the master context acted as an implicit contract more effective than any prose.
2. **Data-schema fidelity.** The generated Prisma schema is essentially interchangeable with the one in `tradingGroups`: same entities (Persona, Gruppo, Portafoglio, Transazione, holdings, friendship, group membership/invitations, watchlist, portfolio history), same enums, correct and systematic use of `Decimal` for monetary values (never `Float`). The “DECIMAL, never float” rule — repeated in the specification, the master context and the individual features — was respected everywhere. **Targeted redundancy on critical rules pays off.**
3. **Functional coverage of the frontend.** The Next.js frontend covers every page in the specification: public homepage, login/registration, personal area, stock page with trading panel, groups area with detail page, social page, person overview, account settings. The dynamic routes (`groups/[id]`, `stock/[id]`, `user/[id]`) are correct, the Pages Router ban was respected, Zustand and React Query are used as specified, and smart polling of pending orders exists (`useSmartPolling.ts`).
4. **Process discipline.** The 384 commits are genuinely Conventional Commits, genuinely granular (one green cycle $\approx$ one commit), with a `chore`: baseline at the start of each feature. The git history is readable and allows the construction order to be reconstructed — it is precisely thanks to this discipline that this analysis was possible.
5. **DB-level triggers and constraints (partial).** The migrations contain real triggers: friendship auto-ordering and uniqueness, no self-friendship, automatic invitation deletion on group join, single-owner, check constraints on Buy/Sell coherence. The QA cycle for document 1 added, during “gap closure”, the trading-permission trigger and the anti-bypass checks with `IS NOT NULL`.
6. **The QA cycle found real problems.** The document 1 diagnostic report correctly identified: missing triggers, an IDOR risk on order creation (no portfolio-ownership verification), an Admin/Owner RBAC divergence on balance changes, an edge case on cancelling sell orders. The first corrections were applied and tested. Phase 3 (doc 4 feature coherence) also intercepted the `GroupCard` triplication *before* implementation, producing `SHARED_CONTRACTS.md`.
7. **The final TODO as an X-ray.** `TODO.md` — generated in the final alignment phase — is a high-quality document: it precisely lists mocks to replace, wrong endpoints, missing client functions, architectural violations (charts that should have been delegated to `TradingView`), logic bugs (average-price loss when cancelling a sale). Almost everything was then actually fixed and ticked off.

6 What went wrong

6.1 The triple implementation of the business logic

This is the most serious and most instructive failure. The project contains **three parallel implementations of the same concepts** (Table 3):

The two “Portfolio” services share nothing: one uses `Prisma.Decimal` and the error codes from `types/errors.ts`, the other uses `decimal.js` and its own error hierarchy (`PortfolioServiceErrors`), with invented concepts such as `DEFAULT_GROUP_BUDGET_LIMIT`. Neither is wrong per se — they are *coherent but incompatible worlds*, built by agents who had been ordered to ignore anything outside

Origin	Convention	Artifacts
Doc 1 features (database)	Italian, <code>nome.service.ts</code>	<code>portfolio.service.ts</code> , <code>transazione.service.ts</code> , 11 repositories in <code>src/repositories/</code> (<code>persona</code> , <code>portafoglio</code> , ...)
Doc 2 features (backend)	English, <code>PascalCase.ts</code>	<code>PortfolioService.ts</code> , <code>TradingService.ts</code> , 4 repositories in <code>src/database/repositories/</code>
Final wiring (TODO phase)	Inline, direct Prisma	<code>runtimeDependencies.ts</code> : 1,317 lines re-implementing orders, balances and snapshots with direct Prisma queries

Table 3: The three coexisting implementations of the same logic.

their own feature file. The problem is acknowledged internally too: the last open item of `TODO.md` reads “*There is redundant or non-unified logic between `TransazioneService.executeOrder` and `TransactionWorker.ts`*”.

6.2 The queue architecture: built, tested, never wired

Features 1–3 of document 2 built the whole BullMQ/Redis machinery: `src/queue/` (connection, queues, scheduler, types), `TransactionWorker.ts` (366 lines), `SnapshotWorker.ts`, with green unit tests and mocks compliant with the master-context rule. Then, at real-wiring time, `runtimeDependencies.ts` implemented the trading engine with two plain `setInterval` calls (order sweep and snapshots), directly invoking `TransazioneService.executeOrder`. Result: **the entire queue infrastructure is dead code** — no runtime file imports the workers or the queues.

The irony is that the `setInterval` choice is the *right* one: it is exactly what `tradingGroups` does (an internal 60-second cron in `tradingEngine.ts`). The problem is not the final solution, but that the pipeline had an infrastructure built and tested for days which the integration then ignored, without anyone removing or wiring it.

6.3 The API monolith and the app that ran on mocks

`createApiApp.ts` is a **2,553-line** file containing all 68 endpoints. It defines its own dependency contract (`ApiDependencies`: `authService`, `portfolioService`, `tradingService`, `marketService`, `adminService`) which **matches none of the services built by the doc 1 and doc 2 features**: it is a third taxonomy, invented by the “API routes” feature agent. To make it run, `src/index.ts` provided `createDevDependencies()`: hardcoded fake data (a portfolio holding AAPL and MSFT, a `dev-access-token`, searches returning empty arrays).

In practice, **for most of the project’s life the app “worked” on fake data**, and all HTTP tests passed against the mocks. Only in the final phase (`TODO.md`, item 1) were the mocks replaced by the real Prisma bootstrap — and it is in this late welding that the chaos concentrated: the wiring phase had to discover and fix dozens of discrepancies in one go (wrong paths, missing client methods, unwired cron, snapshots ignoring frozen assets).

6.4 The breakdown of the failing tests

The 81 failing suites split into two very different families:

Family	Suites	Cause
Doc 1 feature tests (friendship, watchlist, user, transaction, stock, portfolio, permission, membership, invitation, holdings, historical, group: 6–7 suites each)	~73	<code>PrismaClientInitializationError</code> : the tests require a real PostgreSQL with local credentials (868 “Authentication failed against database server” errors). Not reproducible outside the machine/moment where they were written.
Frontend and integration tests (homepage it. 1, personal-dashboard it. 1/8/11, social-interface it. 5, doc4-remediation)	~8	Real regressions : later wiring and remediation commits (e.g. 6b83485 “close feature3 wiring gaps”) modified components — removed testids such as <code>portfolio-overview-skeleton</code> , changed compositions — without re-running the earlier iterations’ suites.

Table 4: Classification of the 81 failing suites.

Both families are *pipeline* failures, not bad luck:

- The first is a **direct violation of the most emphasized rule of the master context** (“MANDATORY MOCKING [...] tests must NEVER attempt real network connections”): the document 1 agents systematically wrote integration tests against a live Postgres. The upper-case rule was not enough, because their feature files (their “universe of knowledge”) described triggers and DB constraints that *only make sense* on a real database: when the local instruction and the global rule conflict, the agent follows the local instruction.
- The second is the absence of a **regression gate**: no agent in later phases was required to re-run the whole suite before committing. Each one verified only its own tests (“green”), and other people’s breakages remained invisible. It is the perfect formalization of context rot: *whatever is outside the agent’s context does not exist, not even when it is a red test*.

6.5 RBAC: the middleware exists, the routes do not use it

Feature 2 of document 3 built `rbacGroupMiddleware.ts` + `PermissionSystem` with green tests. The routes in `createApiApp.ts`, however, perform manual role checks (`getEffectiveGroupRole`) — still an open TODO item. Same pattern as the queue: **the right component exists, the integration ignores it**. The “Admin kicks Admin” hole was later closed, but in the service layer, creating yet another source of truth about permissions (middleware, service, inline checks).

6.6 Documentation inflation and invented features

The ban on summarizing in phase 1 turned 327 lines of specification into 11,154 lines of documents. Within that 34× expansion, each document *invented* plausible but never-requested requirements, which later phases treated as contractual obligations, producing entire features:

- doc 2, feature 11: *Performance Monitoring* (`PerformanceCollector`, `PerformanceAnalytics`, `AlertManager`, `HealthCheckService`);
- doc 3, features 9–10: *Error Handling & Resilience* (circuit breaker, retry with backoff) and *Rate Limiting & Security* (`ddosProtector`, `securityAuditLogger`, adaptive rate limiter);
- doc 4, feature 7: user achievements, privacy settings, performance charts with timeframes — absent from the specification;

- the master context added, on its own, the dual-API strategy “Finnhub real-time + Polygon.io daily” (the specification mentioned Finnhub *or* Polygon as alternatives), and an `AnalyticsWebSocketService` appeared in the code even though the specification explicitly prescribed short polling.

A substantial share of the ~50,000 generated lines — plausibly a quarter — serves requirements that do not exist. Every invented line cost generation, testing and QA time, and enlarged the surface on which context rot could act.

6.7 Anomalous project structure

A single `package.json` mixes Express, Prisma, BullMQ, Next.js, React 19 and the testing libraries; the frontend lives in `src/frontend` *inside* the backend source tree (with the `.next` folder committed alongside the sources); tests sit in three different places (`src/tests/`, `*.test.ts` next to the files, `__tests__/`). This is the accumulation of 42 local decisions never reconciled — no agent had the authority (or the context) to say “the frontend is a separate project”. `tradinGroups`, by comparison, has two independent packages with clean boundaries.

6.8 Uneven QA coverage

The diagnostic cycle foreseen by phase 6 is documented only for document 1: `reports/` contains only `REPORT_DIAGNOSTICO_DOC1_GAP_ANALYSIS.md` (with two gap-closure cycles). For documents 2, 3 and 4 no equivalent reports exist in the repository. Precisely the areas without formal QA (API wiring, frontend) are where the real regressions concentrate.

7 Comparison with tradinGroups

7.1 Opposite philosophies

	tradinGroups (manual)	AppFromPrompts (generated)
Philosophy	The bare minimum that works	The theoretical maximum described in the documents
Trading engine	<code>setInterval</code> 60s + endpoints protected by <code>CRON_SECRET</code>	BullMQ/Redis (dead) + <code>setInterval</code> (active)
Frontend	Vite SPA, React Router, separate package	Next.js App Router inside the backend
Images	Cloudinary with client-side crop	<code>OptimizedImage</code> + generative SVG placeholders (no real provider wired)
Tests	0 (manual verification + <code>API_ROUTES_CHECKLIST.md</code>)	963 (38% failing)
Security	JWT + auth/cron middleware, helmet, rate limit	JWT + RBAC + audit + DDoS protector (partially integrated)
Source of truth	The code itself + a checklist	5 documents + 42 features + master context

Table 5: Architectural and methodological comparison.

7.2 Key observations

1. The spec was more ambitious than the real app — and the pipeline could not know it. `specifich.md` describes Redis, BullMQ, multi-level caching, three WebP image formats. `tradinGroups` implements almost none of that: no Redis, no queues, a simple cron. Whoever wrote the spec knew which parts were aspirational; the pipeline, by construction (“omit nothing”), treated all of them as binding requirements and even amplified them. **A specification handed**

to an AI pipeline must distinguish the “must” from the “nice to have”, because the AI lacks the context to do it on its own.

2. Where the spec was precise, the generated code is aligned. Data schema, exact error messages, pending/executed order flow, private/group isolation: on everything the specification defined in detail, the two projects converge remarkably. Output quality is directly proportional to input precision.

3. The generated project is “more complete” on paper, less functional in practice. The generated project has more endpoints, more declared DB constraints, tests, monitoring, resilience. But `tradingGroups` is an application that runs coherently end-to-end, while the generated one is a set of excellent modules with fragile welds. For a trading simulator, the end-to-end integrity of the order→execution→balance flow is worth more than any circuit breaker.

8 Root-cause analysis: where context rot struck

The symptoms described above stem from five structural causes of the pipeline.

8.1 Cause 1 — Agent isolation without executable shared contracts

The rule “each agent works only with its feature file + master context” was designed precisely to fight context rot: small, fresh, focused contexts. However, it removed the only antidote to divergence: *mutual knowledge*. The master context described business rules and stack, but not the **concrete contracts**: file names, service signatures, naming convention, folder structure. Each agent therefore invented its own taxonomy, and 42 taxonomies do not compose into a system. The counter-proof exists within the project itself: where a shared contract existed (`SHARED_CONTRACTS.md` for doc 4; the Prisma schema for everyone), convergence happened.

8.2 Cause 2 — Integration as a final phase instead of a continuous constraint

The pipeline first built all the pieces and then welded them (mocks in `index.ts` → final wiring). Every interface error remained invisible until all the pieces were stacked on top of each other, and at that point the fix happened under pressure, in a by-then enormous context, by agents who did not know the original features: this is the exact moment when the third inline implementation (6.1) and the frontend regressions were born. In context-rot terms: **the longer integration is postponed, the more context is needed to do it, and the less context is available.**

8.3 Cause 3 — Local “green” mistaken for global “green”

The TDD protocol required committing at every green *of one’s own feature*. No phase required re-running everything. The reports are therefore all sincerely green while the project is globally red. Without an automatic gate external to the agents (CI), self-certification has no probative value: the agent reports what it sees in its context, and in its context everything really was green.

8.4 Cause 4 — Documentation expansion as a hallucination multiplier

Each generative document→document step adds plausible details absent from the source; the next step treats them as requirements. With four hops (specification → documents → features → master context → code), the inventions accumulated and *legitimized one another* (the rate limiting invented by doc 3 flowed back into the master context, which made it mandatory for everyone). This is context rot in documentary form: the source of truth drifts further away with every copy.

8.5 Cause 5 — Global rules losing against local instructions

The mandatory-mocking rule, although written in upper case in the master context, was ignored by 12 out of 12 features of document 1 — because their feature files required verifying triggers and constraints that only exist in a real database. When the global rule contradicts the local task, the agent resolves the conflict in favour of the task. Cross-cutting rules must therefore be made *impossible to violate* (environments, fixtures, provided test scripts) rather than *forbidden in words*.

9 Recommendations for the next iteration of the pipeline

1. **Replace expansion with structuring.** Do not ask for four “long, exhaustive documents that omit nothing”: ask for documents that *reorganize* the specification without adding requirements, with the inverse rule (“if a requirement is not in the specification, it cannot appear; if you believe something is missing, list it in an OPEN QUESTIONS section”). Mark in the specification what is binding and what is aspirational.
2. **Contracts before features (contract-first).** Before launching any implementing agent, generate and freeze: the Prisma schema, the endpoint list with request/response DTOs (e.g. OpenAPI), **the TypeScript interfaces of the services including file names**, the folder structure and the naming convention (one single language). The doc 4 `SHARED_CONTRACTS.md` should be promoted from patch to mandatory phase for the whole system. Every feature implements against these contracts, and compilation becomes a free integration test.
3. **Walking skeleton before features.** First agent: build the minimal real end-to-end app (a user registers, buys a stock, the cron executes it, the balance updates — real DB, zero mocks in the runtime). All subsequent features graft onto a system that already works, and every integration breakage is visible immediately. Explicitly forbid “convenience” dependency injection with hardcoded mocks in the main entry point.
4. **A mechanical regression gate.** Every agent, before every commit, runs *the whole* suite (or at least a centrally defined subset of end-to-end smoke tests), not just its own tests. Ideally: an `npm run verify` script provided by the skeleton, whose green outcome is a precondition for the commit. Self-certification in the report must be replaced by the actual output of the command.
5. **Two test categories, with provided infrastructure.** Separate `test:unit` (mocks, always runnable) from `test:integration` (ephemeral PostgreSQL via Docker/testcontainers, started by the script itself). The rule “no real connections in unit tests” is enforced by providing the setup template, not by writing it in upper case.
6. **A recurring integrator agent.** Every N features (not at the end), an agent with one exclusive job: run the full suite, look for symbol/service duplications, verify that every new module is imported by the runtime (code unreachable from `index.ts` is an error), update the contracts if needed. It would have intercepted both the double service layer and the dead queue weeks in advance.
7. **Uniform, tracked QA.** The gap-analysis cycle must be run (and archived in `reports/`) for every document, not just the first one; and every post-QA fix must go back through the regression gate, because it is statistically the phase that introduces the most regressions.
8. **Size features by value, not by decomposability.** 42 features for this app mean 41 seams. A decomposition into 10–15 *vertical slices* (e.g. “complete order lifecycle, DB→API→UI”) reduces the internal interfaces, which are exactly where context rot strikes.

9. **The master context must contain normative examples, not just rules.** One example file per service, test and route (with naming, imports, structure) constrains more than ten rules in prose: agents imitate far better than they obey.

10 Conclusions

The pipeline demonstrated that a team of agents can produce, in a few days, the complete surface of a non-trivial application with high local quality: correct types, faithful data schema, complete UI, exemplary git history. It also demonstrated, with quantitative evidence, that **local quality does not automatically compose into global quality**: 38% of tests failing, three implementations of the same logic, an entirely dead queue infrastructure and a final integration that broke what used to be green.

Context rot did not strike where it was expected — inside the individual agent’s context, which the small-and-fresh-context strategy actually protected well — but **in the spaces between agents**: in the interfaces never agreed upon, in the requirements multiplied at every documentation hop, in the local “green” never verified globally, in the integration postponed to the last moment. Fixing the pipeline therefore does not require better prompts, but **shared mechanical constraints**: executable contracts defined up front, a working skeleton from day one, and a regression gate that no agent can self-certify. Words in a context decay; compilable contracts and executed tests do not.

A Appendix A — Full test-suite outcome

Run of `npx jest` on the `AppFromPrompts` repository (July 12, 2026):

Test Suites: 81 failed, 1 skipped, 183 passed, 264 of 265 total

Tests: 370 failed, 4 skipped, 589 passed, 963 total

Suites failing due to real regressions (non-environmental):

- `src/tests/homepage/iteration1.homepage-layout.test.tsx`
- `src/tests/personal-dashboard/iteration1.dashboard-layout.test.tsx`
- `src/tests/personal-dashboard/iteration8.dashboard-composition.test.tsx`
- `src/tests/personal-dashboard/iteration11.page-wiring-actions-and-polling.test.tsx`
- `src/tests/social-interface/iteration5.social-dashboard-composition.test.tsx`
- `src/tests/integration/doc4-remediation.integration.test.ts`

Suites failing due to a real-PostgreSQL dependency (868 occurrences of *Authentication failed against database server*): all suites in the folders under `src/tests/` belonging to document 1 features — `friendship-system`, `watchlist-management`, `user-management`, `transaction-processing`, `stock-management`, `portfolio-management`, `permission-management`, `membership-management`, `invitation-system`, `holdings-management`, `historical-data-management`, `group-management` — plus `integration/doc1-full-system` and `integration/gap-closure`.

B Appendix B — Duplication inventory

- **Portfolio services:** `src/services/PortfolioService.ts` (494 lines, `decimal.js`, own errors) vs `src/services/portfolio.service.ts` (118 lines, `Prisma.Decimal`) vs inline logic in `src/runtime/runtimeDependencies.ts`.
- **Trading services:** `src/services/TradingService.ts` (310 lines) vs `src/services/transazione.service.ts` (183 lines) vs `src/workers/TransactionWorker.ts` (366 lines, never imported by the runtime).
- **Repositories:** `src/repositories/` (11 files, Italian naming: `persona`, `portafoglio`, `gruppo`, ...) vs `src/database/repositories/` (4 files, English naming: `UserRepository`, `PortfolioRepository`, ...).
- **Order execution:** `TransazioneService.executeOrder` (used by the runtime) vs `TransactionWorker` (BullMQ queue, dead) — a duplication acknowledged as an open item in `TODO.md`.
- **Permission checks:** `rbacGroupMiddleware` (built, not used by the routes) vs manual `getEffectiveGroupRole` checks in `createApiApp.ts` vs checks in `PermissionService` — three sources of truth.

C Appendix C — Essential timeline of the generated project

Reconstructed from the git history (384 commits in total):

Date	Commits	Event
Feb 22–23	14	Doc 4 features (frontend): user profile, image system; first Next.js skeleton.
Mar 13	18	Resumption after three weeks: friendship system with DB guards (doc 1).
Mar 16	23	Stock, portfolio and transaction management (doc 1).
Mar 17	200	Peak: doc 1 completion, gap analysis and two gap-closure cycles; doc 2 features (queue, workers, services); doc 3 start.
Mar 18	128	Doc 3 features (JWT, RBAC, endpoints, resilience, rate limiting), API integration, <code>TODO.md</code> remediation, real runtime wiring.
Mar 19	1	Last commit (homepage client). On the same day, the manual development of <code>tradingGroups</code> begins.

Table 6: Timeline of the generated repository.